- **语义分析**
- **LR(1)和 LALR**

- **语义分析**

5.1 语法制导定义

语法制导定义（Syntax-Directed Definition，SDD）是一个上下文无关文法和属性及规则的结合。属性和文法符号相关联，而规则和产生式相关联。如果X是一个符号而a是X的一个属性，那么我们用X.a来表示a在某个标号为X的分析树结点上的值。如果我们使用记录或对象来实现这个语法分析树的结点，那么X的属性可以被实现为代表X的结点的记录的数据字段。属性可以有多种类型，比如数字、类型、表格引用或串。这些串甚至可能是很长的代码序列，比如编译器使用的中间语言的代码。

将属性和文法符号关联，规则和产生式关联。

属性分为两类。

综合属性 (synthesized attribute)

- 结点N的属性值由N的产生式所关联的语义规则来定义
- 通过N的子结点或N本身的属性值来定义

继承属性 (inherited attribute)

- 结点N的属性值由N的父结点所关联的语义规则来定义
- 依赖于N的父结点、N本身和N的兄弟结点上的属性值

来通过例子理解下两类属性。

SDD的例子

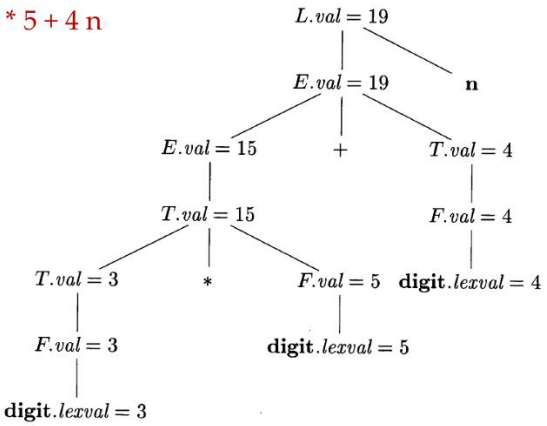
- 计算表达式行L的值 (属性val)
- 计算L的val值需要E的val值，E的val值又依赖于E_1和T的val值，…
- 终结符号digit有综合属性lexval

产生式	语义规则
1) $L \rightarrow E$ **n**	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow$ **digit**	$F.val =$ **digit**.lexval

SDD 目标是得到一个标注的语法分析树，如何计算？来看一个例子理解。

注释分析树的例子

3 * 5 + 4 n



上述例子称为 S 属性的 SDD

S属性的SDD

- 只包含综合属性的SDD称为**S属性**的SDD
  - 每个语义规则都根据产生式体中的属性值来计算头部非终结符号的属性值
- S属性的SDD可以和LR语法分析器一起实现
  - 栈中的状态/文法符号可以附加相应的属性值
  - 归约时，按照语义规则计算归约得到的符号的属性值

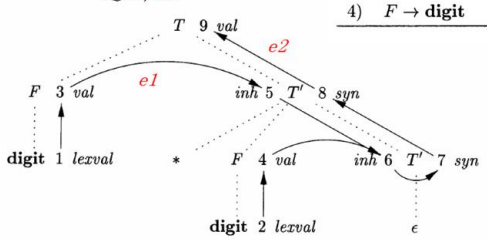
实践中很少先构造语法分析树再进行SDD求值，但在分析树上求值有助于翻译方案的可视化，便于理解

从上述例子的计算中可以感觉出计算顺序。
依赖图描述了某棵特定的分析树上各个属性之间的信息流(计算顺序)– 从实例 a1 到实例 a2 的有向边表示计算 a2 时需要 a1 的值。
通过例子来理解下。

依赖图的例子

- 3 * 2的注释分析树
- $T \rightarrow F\ T'$
  - $T'.inh = F.val$
  - $T.val = T'.syn$
  - 边e1, e2

产生式	语义规则
1) $T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$



属性值的计算顺序 各个属性值需要按照依赖图的拓扑排序 (topological sort) 的顺序进行计算– 如果依赖图中存在环，则属性计算无法进行(无法排序) 。给定一个 SDD，很难判定是否存在一棵分析树， 其对应的依赖图包含环。但是特定类型的 SDD 一定不包含环，且有固定的 计算顺序– 如：S 属性的 SDD，L 属性的 SDD。
S 属性的 SDD 计算较为简单（前者为自顶向下，后者为自底向上）
这里结合上面的例子仔细讲解下。

按照后序遍历的顺序计算属性值即可

```
postorder(N)
{
    for (从左边开始，对N的每个子结点C)
        postorder(C);
    // 递归调用返回时，各子结点的属性已计算完毕
    对N的各个属性求值;
}
```

在LR分析过程中，实际上不需要构造分析树结点

L属性的SDD

- 每个属性
  - 是综合属性，或
  - 是继承属性，且$A \rightarrow X_1X_2...X_n$中计算$X_i.a$的规则只用
    - A的继承属性，或
    - X_i左边的文法符号X_j的继承属性或综合属性，或
    - X_i自身的继承或综合属性 (这些属性间的依赖关系不形成环)
- 特点
  - 依赖图中的边
    - 综合属性从下到上
    - 继承属性从上到下，或从左到右
  - 计算一个属性值时，它所依赖的属性值都已计算完毕

所需的属性都已经计算完毕。

L属性SDD的例子及反例

- 非L属性的例子
  - $A \rightarrow BC$ $A.s = B.b;$ $B.i = f(C.c, A.s)$
- L属性的例子

产生式	语义规则
1) $T \rightarrow F\,T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow *\,F\,T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

结合例子讲解。（chapter5 p33)

L属性SDD和自顶向下语法分析 (2)

- L属性SDD其属性总可以按如下方式计算

```
L_dfvisit(n)
{
    for m = 从左到右n的每个子节点 do
    {
        计算m的继承属性;
        L_dfvisit(m);
    }
    计算n的综合属性;
}
```

语法制导的翻译方案(SDT) 是在产生式体中嵌入 语义动作(程序片断) 的上下文无关文法

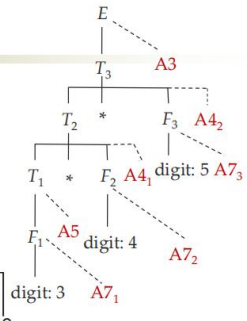
语法制导的翻译方案

- **语法制导的翻译方案** (SDT) 是在产生式体中嵌入语义动作 (程序片断) 的上下文无关文法
- SDT的基本实现方法
  - 建立语法分析树
  - 将语义动作看作是虚拟结点
  - 从左到右、深度优先地遍历分析树 (前序遍历)，在访问虚拟结点时执行相应的语义动作
- 用SDT实现两类重要的SDD (无需建分析树)
  - 基本文法是LR的，且SDD是S属性的
  - 基本文法是LL的，且SDD是L属性的

下面例子中的顺序从自顶向下（前面提到的后序遍历）或者自底向上都可以理解

例子

- 语句3 * 4 * 5的分析树
- 动作执行顺序
  - $A7_1$, A5, $A7_2$, $A4_1$, $A7_3$, $A4_2$, A3
  - 动作的不同实例所访问的属性值属于不同的结点



L	→	E **n**	{ print($E.val$); } A1
E	→	$E_1 + T$	{ $E.val = E_1.val + T.val$; } A2
E	→	T	{ $E.val = T.val$; } A3
T	→	$T_1 * F$	{ $T.val = T_1.val \times F.val$; } A4
T	→	F	{ $T.val = F.val$; } A5
F	→	(E)	{ $F.val = E.val$; } A6
F	→	**digit**	{ $F.val =$ **digit**.$lexval$; } A7

将每个语义动作替换为一个独有的非终结符号 Mi，其产生式为 Mi → ε

如果新的文法可以由某种方法进行分析，那么这个 SDT 就可以在这个分析过程中实现

可在语法分析过程中实现的SDT

- 实现SDT时，实际上并不会真的构造语法分析树，而是在分析过程中执行语义动作
- 即使基础文法可以应用某种分析技术，仍可能因为动作的缘故导致此技术不可应用

- 判断是否可在分析过程中实现
  - 将每个语义动作替换为一个独有的非终结符号M_i，其产生式为$M_i \rightarrow \varepsilon$
  - 如果新的文法可以由某种方法进行分析，那么这个SDT就可以在这个分析过程中实现

后缀分析方案。

后缀翻译方案的例子

- 实现桌上计算器的后缀SDT

L	$\rightarrow$	E **n**	{ print($E.val$); }
E	$\rightarrow$	$E_1 + T$	{ $E.val = E_1.val + T.val$; }
E	$\rightarrow$	T	{ $E.val = T.val$; }
T	$\rightarrow$	$T_1 * F$	{ $T.val = T_1.val \times F.val$; }
T	$\rightarrow$	F	{ $T.val = F.val$; }
F	$\rightarrow$	(E)	{ $F.val = E.val$; }
F	$\rightarrow$	**digit**	{ $F.val =$ **digit**.$lexval$; }

- 注意动作中对属性值的引用
  - 允许语句引用全局变量、局部变量和文法符号的属性
  - 文法符号的属性只能被赋值一次

分析栈实现（p44）

消除左递归时的动作应该怎么考虑？

消除左递归时SDT的转换 (1)

- 如果动作不涉及属性值，可以把动作当作终结符号进行处理，然后消除左递归

- 原始的产生式
  - $E \rightarrow E_1 + T$ { print('+'); }
  - $E \rightarrow T$
- 转换后得到
  - $E \rightarrow T\ R$
  - $R \rightarrow + T$ { print ('+'); } R
  - $R \rightarrow \varepsilon$

这个例子结合书画图理解。思路是先用继承向下算，然后结束时综合往上传。

消除左递归时SDT的转换 (2)

- 如果涉及属性值的计算，则有通用的解决方案

- 假设
  - $A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \}$
  - $A \rightarrow X \{ A.a = f(X.x) \}$
- 那么
  - $A \rightarrow X \{ R.i = f(X.x) \} R \{ A.a = R.s \}$
  - $R \rightarrow Y \{ R_1.i = g(R.i, Y.y) \} R_1 \{ R.s = R_1.s \}$
  - $R \rightarrow \varepsilon \{ R.s = R.i \}$

现在考虑更为一般的 L 属性的 SDD。

L属性的SDT

- 除了通用的SDT实现技术，若基础文法是LL的，则可以将L属性SDD转换成一个SDT，该SDT可以在自顶向下的分析过程中实现

- 从L属性的SDD到SDT的转换
  - 将每个语义规则看作是一个赋值语义动作
  - 将赋值语义动作放到相应产生式$A \rightarrow X_1 X_2 \ldots X_n$的适当位置
    - 计算X_i继承属性的动作插入到产生式体中X_i的左边
    - 计算产生式头A综合属性的动作在产生式的最右边

通过一个例子来理解下。

L属性的SDD的例子

- SDD

$S \rightarrow \textbf{while} (C) S_1$	$L1 = new();$ $L2 = new();$ $S_1.next = L1;$ $C.false = S.next;$ $C.true = L2;$ $S.code = \textbf{label} \parallel L1 \parallel C.code \parallel \textbf{label} \parallel L2 \parallel S_1.code$

- 继承属性
  - *next*：语句结束后应该跳转到的标号
  - *true*, *false*：C为真/假时应该跳转到的标号
- 综合属性*code*表示代码

转换为SDT

- 语义动作
  - (a) $L1$ = new( )和$L2$ = new( )：计算临时值
  - (b) $C.false = S.next$; $C.true = L2$：计算C的继承属性
  - (c) $S_1.next = L1$：计算S_1的继承属性
  - (d) $S.code = \ldots$：计算S的综合属型
- 根据放置语义动作的规则得到如下SDT
  - (b)在C之前，(c)在S_1之前，(d)在最右端
  - (a)可以放在最前面

$S \rightarrow$ **while** (	{ $L1 = new()$; $L2 = new()$; $C.false = S.next$; $C.true = L2$; }
C)	{ $S_1.next = L1$; }
S_1	{ $S.code$ = **label** ‖ $L1$ ‖ $C.code$ ‖ **label** ‖ $L2$ ‖ $S_1.code$; }

最后来看下实现方式。

L属性SDD的实现

- 使用递归下降的语法分析器
  - 每个非终结符号对应一个函数
  - 函数的参数接受继承属性，返回值包含了综合属性
- 在函数体中
  - 首先选择适当的产生式
  - 使用局部变量来保存属性
  - 对于产生式体中的终结符号，读入符号并获取其(经词法分析得到的)综合属性
  - 对于非终结符号，使用适当的方式调用相应函数，并记录返回值

递归下降实现L属性SDD的例子

```
string S(label next) {
    string Scode, Ccode; /* 存放代码片段的局部变量 */
    label L1, L2; /* 局部标号*/
    if ( 当前输入== 词法单元while ) {
        读取输入 ;
        检查 '(' 是下一个输入符号 , 并读取输入 ;
        L1 = new();  C.false  C.true
        L2 = new();  ↙  ↙
        Ccode = C(next, L2);
        检查 ')' 是下一个输入符号 , 并读取输入 ;
        Scode = S(L1);  ←  S.next
        return("label" ‖ L1 ‖ Ccode ‖ "label" ‖ L2 ‖ Scode);
    }
    else /* 其他语句类型 */
}
```

55

- **LR(1)和 LALR**

步骤为先增广文法并算 FIRST 和 FOLLOW，再画图，然后列表，最后按要求剔除。这里直接用一个例子来讲解。画图这一步是核心。

先看一个简单的例子，结合例子讲算法。(Chapter4 p109)

LR(1)项集的CLOSURE算法

- 注意在添加$[B \to \cdot\gamma, b]$时，b的取值范围

SetOfItems CLOSURE(I) {
    repeat
        for (I中的每个项 $[A \to \alpha\cdot B\beta, a]$)
            for (G'中的每个产生式 $B \to \gamma$)
                for (FIRST(βa)中的每个终结符号 b)
                    将$[B \to \cdot\gamma, b]$加入到集合I中;
    until 不能向I中加入更多的项;
    return I;
}

LR(1)项集的GOTO算法

- 和LR(0)项集的GOTO算法基本相同

SetOfItems GOTO(I, X) {
    将J初始化为空集;
    for (I 中的每个项 $[A \to \alpha\cdot X\beta, a]$)
        将项 $[A \to \alpha X\cdot\beta, a]$加入到集合$J$中;
    return CLOSURE(J);
}

LR(1)项集族的构造算法

- 和LR(0)项集族的构造算法相同

课本有误，
此处修正

void *items*(G') {
    将 C 初始化为{CLOSURE({$[S' \to \cdot S, \$]$})}
    repeat
        for (C 中的每个项集 I)
            for (每个文法符号 X)
                if (GOTO(I, X) 非空且不在 C 中)
                    将 GOTO(I, X) 加入 C 中;
    until 不再有新的项集加入到 C 中;
}

LALR分析技术的基本思想

- 寻找具有相同核心的LR(1)项集，并把它们合并成为一个项集
  - 项集的**核心** (core) 就是项的第一分量的集合
    - I_4和I_7的核心都是{ $C \rightarrow d\cdot$ }
    - I_3和I_6的核心{ $C \rightarrow c\cdot C$, $C \rightarrow \cdot cC$, $C \rightarrow \cdot d$ }
- 一个LR(1)项集的**核心**是一个LR(0)项集

- GOTO(I, X)的核心只由I的核心决定，因此被合并项集的GOTO目标也可以合并
  - 这表示合并之后，我们仍可以建立GOTO关系

LALR分析表构造算法

- 步骤
  - 构造得到LR(1)项集族$C = \{ I_0, I_1, \ldots, I_n \}$
  - 对于LR(1)中的每个核心，找出所有具有该核心的项集，并把这些项集替换为它们的并集
  - 令$C' = \{ J_0, J_1, \ldots, J_m \}$为得到的LR(1)项集族
    - 按照LR分析表的构造方法得到ACTION表 (如果存在冲突，则这个文法不是LALR的)
  - GOTO表的构造：设J是一个或者多个LR(1)项集 (包括I_1)的并集，令K是所有和GOTO(I_1, X)具有相同核心的项集的并集，那么GOTO(J, X) = K
- 得到的分析表称为**LALR语法分析表**

习题的例子，上周群中照片是这题的答案，但上周没有来得及讲完。

Please **construct a LR(1) parsing table for the following ambiguous grammar with the additional conditions** that **else** dangles with the closest previous unmatched **if** , **;** has the property of left associative.

S→**if** S **else** S|**if** S|S**;**S|a